SMART HOSPITAL MANAGEMENT & COMMUNICATION SYSTEM

Abstract

The Smart Hospital Management & Communication System is a web-based application designed to improve hospital administration and communication among Admin, Doctor, Nurse, and Patient users. It provides role-based dashboards for user management, appointments, patient information, doctor instructions, nursing updates, and patient requests. The frontend uses HTML, CSS, and JavaScript. Python FastAPI provides REST APIs that exchange JSON over HTTP, while SQLite stores application data. Python TCP socket programming is included for real-time messages and emergency alerts. The project follows the Software Development Life Cycle, including planning, requirements analysis, design, implementation, testing, deployment, and maintenance.

Table of Contents

1. Introduction
2. Literature Survey
3. SDLC and Project Planning
4. Requirements Analysis
5. System Design
6. Frontend–Backend and TCP Communication
7. Implementation
8. Testing
9. Results and Discussion
10. Security, Limitations and Risk Management
11. Deployment and Maintenance
12. Conclusion and Future Scope
13. References

Chapter 1: Introduction

1.1 Background

Hospitals coordinate patient registration, appointments, medical updates, nursing care, and communication. Manual or disconnected processes can cause delays, duplicated records, and difficulty accessing information. A centralized digital system can improve coordination and accessibility.

1.2 Problem Statement

Hospitals require a unified platform to manage users, appointments, patient information, doctor instructions, nurse updates, and urgent communication. The proposed system addresses these needs through role-based dashboards, REST APIs, SQLite, and TCP-based communication.

1.3 Objectives

- Create secure login and role-based access.
- Develop separate dashboards for Admin, Doctor, Nurse, and Patient.
- Manage doctors, nurses, patients, and appointments.
- Allow doctors to update patient status and message nurses.
- Allow nurses to update patient condition and send emergency alerts.
- Allow patients to book appointments, view doctors and basic records.
- Demonstrate REST, JSON, HTTP, TCP and Python socket programming.

1.4 Scope

The system covers authentication, user management, appointments, patient-related information, role-specific dashboards, internal communication, and emergency alerts. Video consultation, AI diagnosis, cloud hosting, and external hospital integration are future enhancements.

1.5 Report Organization

The report explains the project through the SDLC process, from planning and requirements to design, implementation, testing, deployment, and maintenance.

Chapter 2: Literature Survey

Existing hospital systems provide features such as registration, appointments, electronic records, billing, and staff management. Smaller organizations may still depend on paper records or informal communication. This project combines hospital management functions with role-based access and TCP-based real-time communication.

Approach	Advantages	Limitations
Manual records	Easy to begin	Slow retrieval, paper loss, duplication
Basic hospital software	Digital records and appointments	Limited communication or role workflows
Cloud platforms	Remote access and scalability	Cost, connectivity and privacy concerns
Proposed system	Role dashboards, REST APIs and TCP alerts	Requires development and maintenance

Chapter 3: SDLC and Project Planning

The Software Development Life Cycle is a structured process for planning, building, testing, deploying, and maintaining software. This project follows an iterative SDLC approach.

Phase	Activities	Deliverables
Planning	Problem, stakeholders, scope, feasibility	Project plan
Requirements	Functional and non-functional analysis	SRS
Design	Architecture, UML, database, network	Design diagrams
Implementation	Frontend, FastAPI, SQLite, sockets	Working modules
Testing	Functional, integration, security and network tests	Test report
Deployment	Configure and run application	Running system
Maintenance	Bug fixes and improvements	Updated versions

3.1 Sprint Plan

Sprint	Tasks
1	Database, project structure, login and authentication
2	Admin, Doctor, Nurse and Patient dashboards
3	Appointments and patient records
4	REST integration, TCP messaging and alerts
5	Testing, documentation and bug fixing

Chapter 4: Requirements Analysis

4.1 Users and Responsibilities

User	Responsibilities
Admin	Add/remove doctors and nurses, register patients, manage appointments, view activity
Doctor	Login, view patients and appointments, update patient status, message nurses, receive alerts
Nurse	Login, view assigned patients, update condition, receive doctor instructions, send alerts
Patient	Register/login, view doctors, book appointments, view status and basic records, send requests

4.2 Functional Requirements

- Authenticate users and identify their roles.
- Display separate dashboards.
- Allow Admin to manage doctors, nurses, patients and appointments.
- Allow doctors to view assigned patients and update status.
- Allow doctors to send instructions to nurses.
- Allow nurses to update patient condition and send emergency alerts.
- Allow patients to book appointments and view appointment status.
- Provide REST APIs using FastAPI and JSON.
- Support TCP socket connections for real-time communication.
- Allow secure logout.

4.3 Non-Functional Requirements

Requirement	Description
Security	Authentication, authorization and protected records
Usability	Simple role-specific screens
Performance	Reasonable response time on local network
Reliability	Validation and error handling
Maintainability	Modular code and documentation
Scalability	Ability to add features and roles

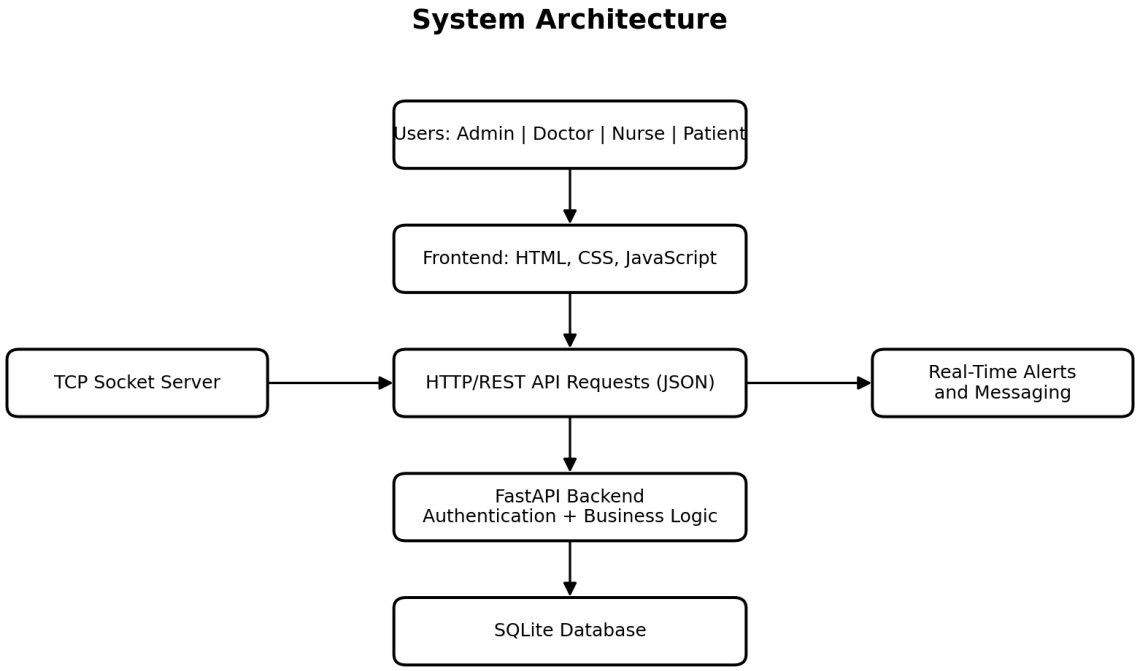
4.4 Hardware and Software Requirements

Category	Requirement
Processor	Intel i3 or equivalent
RAM	4 GB minimum
Storage	10 GB free
Frontend	HTML, CSS, JavaScript
Backend	Python, FastAPI
Database	SQLite
Networking	Python sockets and TCP
IDE	Visual Studio Code
OS	Windows/Linux

Chapter 5: System Design

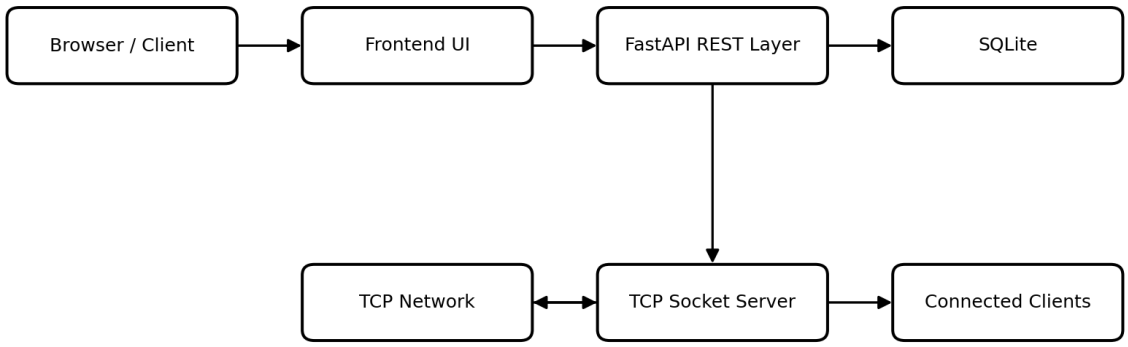
5.1 System Architecture

Users access the frontend through a browser. JavaScript sends HTTP requests to FastAPI. FastAPI validates credentials, applies business logic, and reads or writes SQLite. A separate TCP socket service handles real-time messages and emergency alerts.



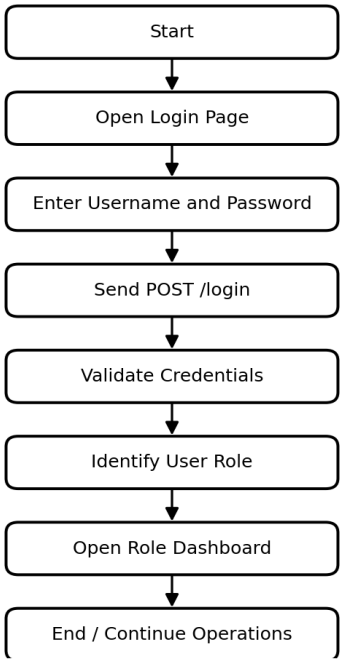
5.2 Block Diagram

Frontend–Backend–TCP Communication Block Diagram

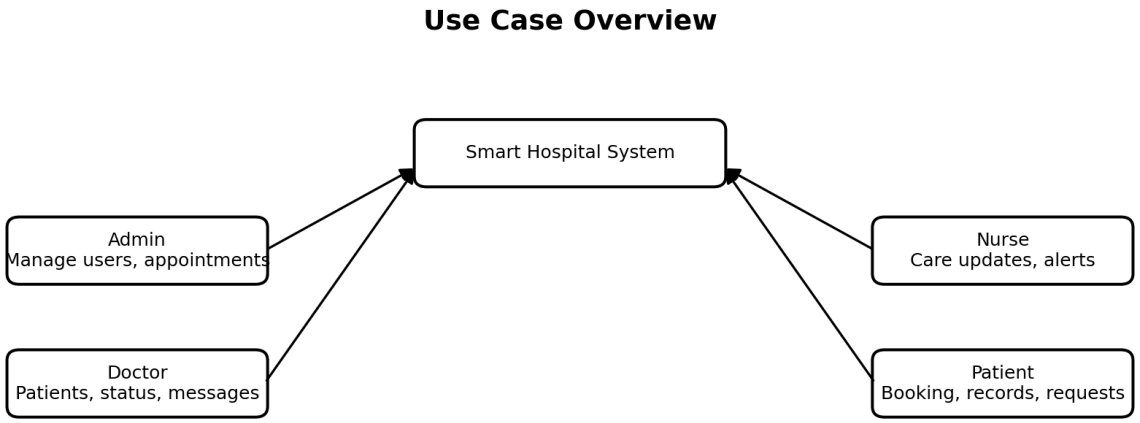


5.3 Login Flowchart

Login and Role-Based Access Flow



5.4 Use Case Diagram



5.5 Database Design

Table	Fields / Purpose
users	id, username, password_hash, role
doctors	id, user_id, specialization
nurses	id, user_id, department
patients	id, user_id, contact and profile
appointments	patient_id, doctor_id, date, status
medical_records	patient_id, doctor_id, notes, status
messages	sender_id, receiver_id, message, timestamp
alerts	sender_id, patient_id, severity, status

5.6 Main Entities

The principal entities are User, Admin, Doctor, Nurse, Patient, Appointment, MedicalRecord, Message, and Alert. User contains common login information while role-specific entities provide specialized operations.

Chapter 6: Frontend–Backend and TCP Communication

6.1 Frontend to Backend Communication

The frontend displays forms, dashboards, tables, buttons, and messages. It does not directly access SQLite. JavaScript uses the fetch() function to send HTTP requests to FastAPI endpoints. FastAPI receives and validates the request, performs the operation, and returns a JSON response. JavaScript reads the response and updates the webpage.

- User enters login details.
- JavaScript collects the form values.
- A POST request is sent to /login.
- FastAPI validates credentials and identifies the role.
- FastAPI returns JSON containing status and role.
- The frontend redirects the user to the correct dashboard.
- Dashboard actions generate additional GET, POST, PUT or DELETE requests.

6.2 Example REST Request and Response

POST /login
Content-Type: application/json

{"username":"doctor1","password":"example-password"}

Response:
{"status":"success","role":"doctor","message":"Login successful"}

6.3 API Table

Method	Endpoint	Purpose
POST	/login	Authenticate user
POST	/register	Register patient or permitted user
GET	/doctors	List doctors
GET	/patients	List authorized patients
POST	/appointments	Book appointment
GET	/appointments	View appointments
PUT	/appointments/{id}	Update appointment status
PUT	/patients/{id}/status	Update patient status
POST	/messages	Create stored message
GET	/records/{patient_id}	View authorized records

6.4 TCP Socket Programming

TCP is a connection-oriented transport protocol. It establishes a connection before data transfer and provides reliable, ordered delivery of bytes. In this project, a Python socket server can accept authorized clients and forward real-time messages or emergency alerts.

6.5 TCP Working Steps

- Server creates a socket.
- Server binds an IP address and port.
- Server listens for connections.
- Client connects to the server.
- Server accepts the connection.

- Client sends JSON encoded as bytes.
- Server validates and routes the message.
- Receiving client gets the message or alert.
- Connection remains open for multiple messages or closes when finished.

6.6 REST vs TCP

Feature	REST over HTTP	TCP Socket
Purpose	Web operations and APIs	Real-time messages and alerts
Style	Request-response	Persistent connection possible
Data	JSON over HTTP	Bytes; JSON may be encoded
Browser use	Directly supported	Requires bridge or WebSocket-capable layer
Reliability	HTTP commonly runs over TCP	TCP provides ordered reliable delivery

6.7 Important Browser Limitation

A browser cannot normally open an arbitrary raw TCP socket directly. Therefore, the recommended design is to use FastAPI for browser communication and a Python TCP service for real-time communication. FastAPI may authenticate users and bridge approved messages to the socket service. If browser-native real-time communication is required, WebSockets can be considered.

Chapter 7: System Implementation

7.1 Modules

Module	Functions
Authentication	Login, logout, password verification and role identification
Admin	Manage users, appointments and activity
Doctor	Patients, appointments, status, instructions and messages
Nurse	Assigned patients, condition updates and alerts
Patient	Registration, doctors, booking, records and requests
REST API	HTTP requests, JSON validation and responses
Database	Persistent storage of hospital information
TCP Service	Real-time messages and emergency alerts

7.2 Suggested Project Structure

```
smart-hospital/
├── main.py
├── database.py
├── models.py
├── schemas.py
├── auth.py
├── socket_server.py
├── requirements.txt
├── hospital.db
├── templates/
│   ├── login.html
│   ├── admin_dashboard.html
```

```
│   ├── doctor_dashboard.html
│   ├── nurse_dashboard.html
│   └── patient_dashboard.html
└── static/
    ├── style.css
    └── script.js
```

7.3 Error Handling

- Use suitable HTTP status codes.
- Validate missing or incorrect input.
- Reject unauthorized access.
- Display clear frontend errors.
- Handle disconnected TCP clients without crashing.

Chapter 8: Testing

Testing verifies individual functions and the complete integrated system.

ID	Test Case	Expected Result
TC01	Valid admin login	Admin dashboard opens
TC02	Valid doctor login	Doctor dashboard opens
TC03	Valid nurse login	Nurse dashboard opens
TC04	Valid patient login	Patient dashboard opens
TC05	Invalid password	Error displayed
TC06	Admin adds doctor	Record stored
TC07	Patient books appointment	Appointment created
TC08	Doctor views patients	Only permitted records shown
TC09	Nurse updates condition	Update stored
TC10	Doctor sends message	Intended nurse receives message
TC11	Nurse sends emergency alert	Alert delivered and logged
TC12	Unauthorized access	Request rejected
TC13	TCP client disconnects	Server continues
TC14	Logout	Session/token cleared

8.1 Testing Types

- Unit testing
- Integration testing
- Functional testing
- System testing
- Security testing
- Network testing
- User acceptance testing

Chapter 9: Results and Discussion

The expected result is a working prototype with four role-specific dashboards, centralized hospital information, REST API integration, database operations, and TCP socket communication.

9.1 Screenshots to Include

- Login page
- Admin dashboard
- Doctor dashboard
- Nurse dashboard
- Patient dashboard
- Appointment screens
- Patient status and condition screens
- REST API test results
- TCP server and client console output
- Database records

9.2 Benefits

- Reduced manual paperwork.
- Improved information organization.
- Clear role-based permissions.
- Faster doctor–nurse coordination.
- Patient access to appointment and record information.
- Demonstration of web and networking technologies.

Chapter 10: Security, Limitations and Risk Management

10.1 Security

- Hash passwords.
- Apply role-based authorization.
- Validate all input.
- Restrict patient records.
- Avoid hardcoded secrets.
- Use HTTPS in production.
- Authenticate TCP clients.

10.2 Limitations

- SQLite is suitable for a prototype but may not support large hospital concurrency.
- No direct integration with real hospital systems.
- Raw TCP requires authentication, message framing and error handling.
- The system is not a medical diagnosis tool.
- Emergency alerts do not replace official emergency procedures.

10.3 Risks and Mitigation

Risk	Mitigation
Unauthorized access	Authentication and role checks
Data loss	Backups and validation

Network failure	Retries, status checks and logging
Socket crash	Exception handling and cleanup
Scope expansion	Prioritize core modules

Chapter 11: Deployment and Maintenance

11.1 Local Deployment

- Install Python.
- Create and activate a virtual environment.
- Install FastAPI and Uvicorn.
- Configure SQLite.
- Start FastAPI using Uvicorn.
- Start the TCP server separately.
- Open the application in a browser.
- Test all modules.

Typical commands:

```
python -m venv venv
venv\Scripts\activate
pip install fastapi uvicorn
uvicorn main:app --reload
python socket_server.py
```

11.2 Maintenance

- Fix defects.
- Update dependencies carefully.
- Back up the database.
- Review permissions.
- Improve UI from feedback.
- Monitor API and TCP logs.
- Add features after security review.

Chapter 12: Conclusion and Future Scope

12.1 Conclusion

The Smart Hospital Management & Communication System provides a structured digital platform for managing users, appointments, patient information, and communication. By combining HTML, CSS, JavaScript, FastAPI, SQLite, REST, JSON, HTTP, TCP, and Python socket programming, the project demonstrates a complete software system. Following SDLC makes development systematic and supports future maintenance.

12.2 Future Scope

- Cloud deployment.
- Mobile application.
- Email and SMS notifications.
- WebSocket-based browser-native real-time communication.
- Video consultation.
- Laboratory and pharmacy integration.
- Analytics dashboards.
- Multi-factor authentication and audit logging.

Chapter 13: References

- FastAPI Documentation: https://fastapi.tiangolo.com/
- Python Documentation: https://docs.python.org/3/
- Python Socket Documentation: https://docs.python.org/3/library/socket.html
- SQLite Documentation: https://www.sqlite.org/docs.html
- MDN Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- RFC 9293 – Transmission Control Protocol: https://www.rfc-editor.org/rfc/rfc9293

Appendix: Complete Data Flow

A user opens the login page. JavaScript sends credentials to FastAPI through HTTP. FastAPI validates credentials and returns JSON. The frontend opens the correct dashboard. Dashboard actions generate REST requests. FastAPI checks permissions and accesses SQLite. For real-time communication, the authorized service sends messages through TCP. The receiving client gets the message or emergency alert, and important communication may also be stored in the database.